

Como Gerar Token de Acesso Pessoal no GitHub

Passo a Passo para Criar o Token

1. Acessar as Configurações do GitHub

1. Entre na sua conta do GitHub
2. Clique na sua foto de perfil no canto superior direito
3. Selecione Settings

2. Navegar até a Seção de Tokens

1. No menu lateral esquerdo, clique em "Developer settings"
2. Selecione "Personal access tokens"
3. Clique em "Tokens (classic)"
4. Botão "Generate new token" → "Generate new token (classic)"

3. Configurar o Token

Detalhes do token:

- Note: "VS Code" ou um nome descritivo
- Expiration: Recomendo 90 dias ou 1 ano
- Scopes (permissões): Marque pelo menos:
 - ☒ repo (acesso completo aos repositórios)
 - ☒ workflow (se usar GitHub Actions)
 - ☒ write:packages (se publicar pacotes)
 - ☒ delete_repo (se precisar excluir repositórios)

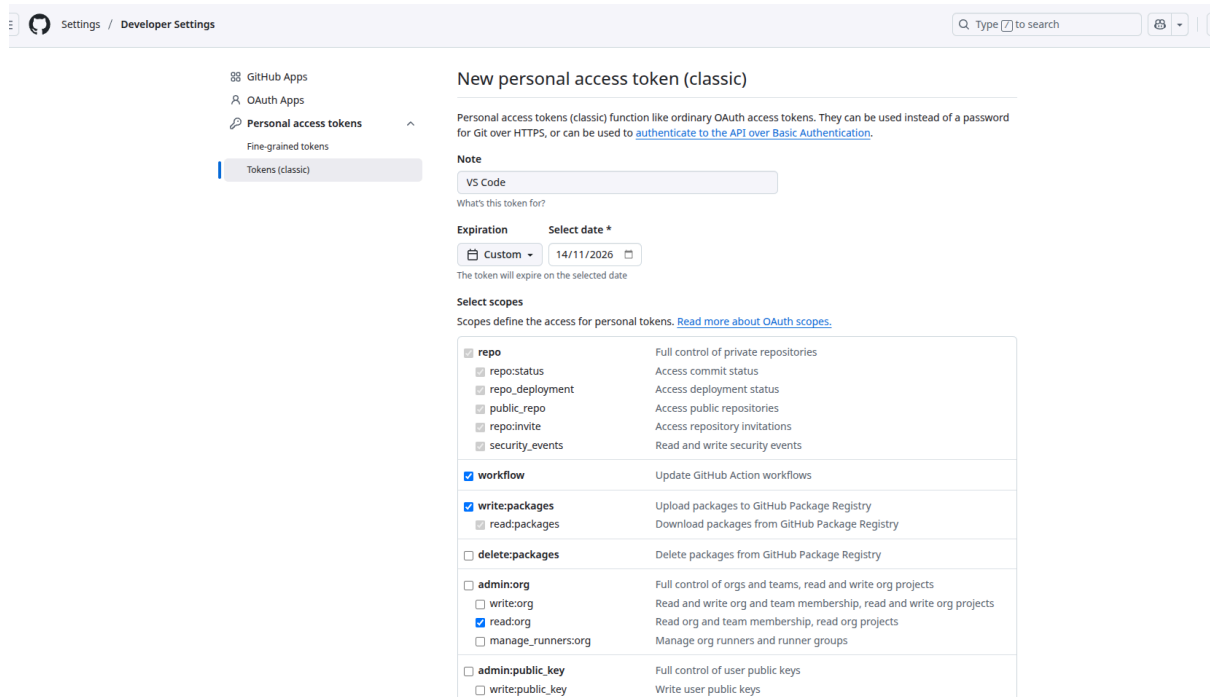
4. Permissões Recomendadas para VS Code

text

- ✓ repo (todas as permissões)
- ✓ workflow
- ✓ write:packages
- ✓ read:org
- ✓ user (todas as permissões)
- ✓ delete_repo

5. Gerar e Salvar o Token

1. Clique em "Generate token"
2. ⚠️ **IMPORTANTE:** Copie o token imediatamente!
3. Cole em um local seguro (gerenciador de senhas)



The screenshot shows the GitHub Developer Settings page for creating a new personal access token (classic). The left sidebar shows the navigation menu with 'Personal access tokens' selected. The main content area is titled 'New personal access token (classic)' and includes a note, a text input for the token name, an expiration date selector, and a list of scopes to be granted.

Settings / Developer Settings

GitHub Apps
OAuth Apps
Personal access tokens
Fine-grained tokens
Tokens (classic)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

VS Code

What's this token for?

Expiration **Select date ***

Custom 14/11/2026

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☒ workflow	Update GitHub Action workflows
☒ write:packages	Upload packages to GitHub Package Registry
☒ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☒ read:org	Read org and team membership, read org projects
☐ manage_runners:org	Manage org runners and runner groups
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys

☐ read:public_key	Read user public keys
☐ admin:repo_hook	Full control of repository hooks
☐ write:repo_hook	Write repository hooks
☐ read:repo_hook	Read repository hooks
☐ admin:org_hook	Full control of organization hooks
☐ gist	Create gists
☐ notifications	Access notifications
☒ user	Update ALL user data
☒ read:user	Read ALL user profile data
☒ user:email	Access user email addresses (read-only)
☒ user:follow	Follow and unfollow users
☒ delete_repo	Delete repositories
☐ write:discussion	Read and write team discussions
☐ read:discussion	Read team discussions
☐ admin:enterprise	Full control of enterprises
☐ manage_runners:enterprise	Manage enterprise runners and runner groups
☐ manage_billing:enterprise	Read and write enterprise billing data
☐ read:enterprise	Read enterprise profile data
☐ scim:enterprise	Provisioning of users and groups via SCIM
☐ audit_log	Full control of audit log
☐ read:audit_log	Read access of audit log
☐ codespace	Full control of codespaces
☐ codespace:secrets	Ability to create, read, update, and delete codespace secrets
☐ copilot	Full control of GitHub Copilot settings and seat assignments
☐ manage_billing:copilot	View and edit Copilot Business seat assignments
☐ write:network_configurations	Write org hosted compute network configurations
☐ read:network_configurations	Read org hosted compute network configurations
☐ project	Full control of projects
☐ read:project	Read access of projects

Usar o Token no VS Code

Método 1: Durante o Login

1. No VS Code, Command Palette (Ctrl+Shift+P)
2. GitHub: Sign In
3. Escolha "Sign in with GitHub using Personal Access Token"
4. Cole o token gerado

Método 2: No Terminal/Command Line

```
bash
```

```
git config --global user.password "seu-token-aqui"
```

Método 3: No VS Code Settings

1. Ctrl+, (abrir configurações)
2. Buscar por "github token"
3. Adicionar nas configurações

Dicas de Segurança



Boas Práticas:

- Nunca compartilhe seu token
- Revogue tokens não utilizados
- Use datas de expiração curtas
- Não commit tokens no código
- Use variáveis de ambiente quando possível



O que Fazer se o Token Vazar:

1. Imediatamente: revogar o token
2. Gerar um novo token
3. Atualizar onde estava sendo usado

Gerenciar Tokens Existentes

Ver/Revogar Tokens:

1. GitHub Settings → Developer settings → Personal access tokens
2. Veja a lista de tokens ativos
3. Clique no ✎ para editar ou 🗑 para revogar

Renovar Token Expirado:

1. Siga os mesmos passos para criar um novo
2. Atualize no VS Code e outros lugares onde usa

Exemplo de Uso no Git

```
bash
```

```
# Configurar git para usar token
```

```
git remote set-url origin https://seu-token@github.com/usuario/repositorio.git
```

```
# Ou na clonagem inicial
```

```
git clone https://seu-token@github.com/usuario/repositorio.git
```

Caso ocorra o erro abaixo ao fazer commits:

Author identity unknown

*** Please tell me who you are.

Run

```
git config --global user.email "you@example.com"
```

```
git config --global user.name "Your Name"
```

to set your account's default identity.

Omit --global to set the identity only in this repository.

Utilize este procedimento para se identificar no GitHub:

O Git precisa saber quem você é para registrar seus commits. Configure sua identidade:

Configuração Global (recomendado)

bash

Configure seu email

```
git config --global user.email "seu-email@example.com"
```

Configure seu nome

```
git config --global user.name "Seu Nome"
```

Verificar as configurações

```
git config --global --list
```

Exemplo prático:

bash

```
git config --global user.email "wellington@example.com"
```

```
git config --global user.name "Wellington"
```

Após configurar, faça o commit novamente:

bash

```
git commit -m "organização do readme"
```

Se quiser configurar apenas para este repositório:

bash

Sem o --global (só vale para este projeto)

```
git config user.email "wellington@example.com"
```

```
git config user.name "Wellington"
```

Dica adicional:

Se você usa GitHub, use o mesmo email da sua conta GitHub para que os commits sejam vinculados ao seu perfil:

bash

```
git config --global user.email "seu-email-do-github@example.com"
```

```
git config --global user.name "Seu Nome no GitHub"
```

Depois disso, seu commit funcionará normalmente! 🚀